

Code Explanation

Data Transformation Module Explanation

This code is a data transformation module designed to join and transform multiple source tables into a target dataset. The transformation involves selecting specific columns, applying joins, and adding default values.

Overview of the Code

The ``transform_policy_data`` function takes a dictionary of source table DataFrames as input and returns a transformed DataFrame. It extracts the required source tables, performs a series of joins, applies transformations to create target columns, and filters out records with null policy IDs.

Step 1: Extracting Source Tables

The function starts by extracting the required source tables from the input dictionary and aliasing them for easier reference in subsequent operations.

```
tx_request_policy = source_tables["T_TX_REQUEST_POLICY"].alias("trp")
tx_basic = source_tables["T_TX_BASIC"].alias("ttb")
tx_relation = source_tables["T_TX_RELATION"].alias("ttr")
tx_request = source_tables["T_TX_REQUEST"].alias("tr")
tx_dtl_beneficiary_change = source_tables["T_TX_DTL_BENEFICIARY_CHANGE"].alias("ttdbc")
```

Step 2: Joining Source Tables

The code performs a series of joins on the extracted source tables based on specific conditions.

```
joined_df = tx_request_policy.join(
    tx_basic,
    (tx_request_policy["POLICY_ID"] == tx_basic["POLICY_ID"]) &
    (tx_request_policy["REQUEST_ID"] == tx_basic["REQUEST_ID"]),
    "inner"
)
joined_df = joined_df.join(
    tx_request,
    joined_df["REQUEST_ID"] == tx_request["REQUEST_ID"],
    "inner"
)
joined_df = joined_df.join(
    tx_relation,
    (joined_df["TRANSACTION_ID"] == tx_relation["TRANSACTION_ID"]) &
    (tx_relation["OBJECT_TYPE"] == "BENEFICIARY"),
    "left"
)
joined_df = joined_df.join(
    tx_dtl_beneficiary_change,
    joined_df["TRANSACTION_ID"] == tx_dtl_beneficiary_change["TRANSACTION_ID"],
    "left"
)
```

Step 3: Applying Transformations

The joined DataFrame is then transformed by selecting specific columns and applying aliases to match the target dataset schema. Default values are also added for certain columns.

```
result_df = joined_df.select(
    col("trp.POLICY_ID").alias("FNL_POLICY_ID"),
    col("tr.REQUEST_ID").alias("TR_REQUEST_ID"),
    # ... other column selections ...
    lit(DEFAULT_VALUES["DATA_TYPE"]).alias("DATA_TYPE"),
    lit(DEFAULT_VALUES["POLICY_TYPE"]).alias("POLICY_TYPE"),
    lit(DEFAULT_VALUES["MC_CRNCY"]).alias("MC_CRNCY"),
    current_timestamp().alias("CREATE_DATETIME"),
    monotonically_increasing_id().alias("TTDBC_CHANGE_ID")
)
```

Step 4: Filtering Out Records with Null Policy IDs

The final step involves filtering out any records with null policy IDs to ensure data quality.

```
result_df = result_df.filter(col("FNL_POLICY_ID").isNotNull())
```

Step 5: Returning the Transformed DataFrame

The transformed DataFrame is then returned as the result of the `transform\_policy\_data` function.

```
return result_df
```